

Chapter 24:
Multidisciplinary Optimization
Algorithms for Optimization, 2nd ed. (2026)
Kochenderfer & Wheeler

Lecture Notes

June 12, 2026

Chapter Outline

- 1 Introduction
- 2 Disciplinary Analyses
- 3 Interdisciplinary Compatibility
- 4 MDO Architectures Overview
- 5 Multidisciplinary Design Feasible (MDF)
- 6 Sequential Optimization and Reliability Assessment
- 7 Individual Discipline Feasible (IDF)
- 8 Collaborative Optimization (CO)
- 9 Simultaneous Analysis and Design (SAND)
- 10 Numerical Example: Spring-Pendulum
- 11 Summary & Exercises

What is Multidisciplinary Design Optimization?

Multidisciplinary Design Optimization (MDO)

addresses problems that:

- Span *multiple interacting disciplines* (structures, aerodynamics, controls, economics, ...)
- Cannot be solved optimally by optimizing each discipline separately
- Require handling *interdependencies* among disciplines

Core challenges:

- Circular dependencies between analyses
- Heterogeneous solvers owned by different teams
- Large numbers of variables and constraints
- Expensive function evaluations (simulations)

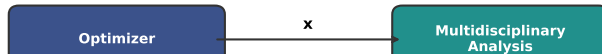
General MDO Formulation

$$\min_{\mathbf{x}} f(\mathcal{A}) \quad \text{s.t. } \mathcal{A} \in \mathcal{X}$$

$$\mathcal{F}_i(\mathcal{A}) = \mathbf{0} \quad \forall i = 1, \dots, m$$

where $\mathcal{A}$ is the **assignment** of all design and response variables.

MDO General Problem Structure



Disciplinary Analyses

Key Definitions

- **Assignment** $\mathcal{A}$: a mapping from variable names to their values.
Contains design variables $\mathbf{x}$ and all response variables $\mathbf{y}^{(1)}, \dots, \mathbf{y}^{(m)}$.
- **Disciplinary analysis** $\mathcal{F}_i$: takes an assignment, runs a discipline-specific computation (e.g. FEA, CFD, simulation), and *updates* the assignment with new response values.
- **Response variables** $\mathbf{y}^{(i)}$: outputs of discipline i that may feed into other disciplines.

Formal Setup

$$\mathbf{y}^{(i)} \leftarrow \mathcal{F}_i(\mathbf{x}, \mathbf{y}^{(-i)})$$

where $\mathbf{y}^{(-i)}$ denotes responses from all other disciplines.

With n design variables and m disciplines each producing n_i responses:

$$|\mathcal{A}| = n + \sum_{i=1}^m n_i$$

Example 24.1 (3-discipline system with $\mathbf{x}$ as single design variable):

$$y^{(1)} = y^{(2)} - x, \quad y^{(2)} = \sin(y^{(1)} + y^{(3)}), \quad y^{(3)} = c$$

Python: Disciplinary Analyses (Example 24.1)

```
import numpy as np

# Each disciplinary analysis takes and updates the assignment dict
def F1(A):
    A['y1'] = A['y2'] - A['x']
    return A

def F2(A):
    A['y2'] = np.sin(A['y1'] + A['y3'])
    return A

def F3(A):
    A['y3'] = np.cos(A['x'] + A['y2'] + A['y1'])
    return A

# Initial assignment for x=1
A = {'x': 1.0, 'y1': 1.0, 'y2': 1.0, 'y3': 1.0}

# Run one pass with ordering F1, F2, F3
A = F1(A); A = F2(A); A = F3(A)
print(A)
```

Key Point

Each function modifies the *shared assignment*

Variable Interdependencies

Interdisciplinary Compatibility

Feasibility Condition

An assignment $\mathcal{A}$ is **interdisciplinarily compatible** if, for all disciplines i :

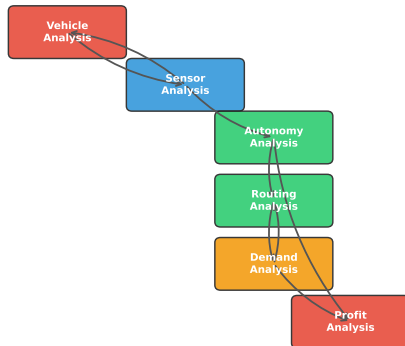
$$\mathcal{F}_i(\mathcal{A}) = \mathcal{A}$$

i.e., running any disciplinary analysis *does not* change the assignment.

Dependency Graph

- Nodes = disciplines (or variables)
- Directed edge $i \rightarrow j$: discipline j requires output of i
- **Acyclic** graph: analyses can be sequenced
- **Cyclic** graph: requires iterative solution (*MDA*)

Ride-Sharing Problem: Dependency Graph



Ride-sharing dependency graph with cycles (vehicle $\leftrightarrow$ sensor, routing $\leftrightarrow$ demand)

Gauss-Seidel MDA Algorithm

Algorithm 24.1: Gauss-Seidel MDA

Input: List of disciplinary analyses $[F_1, \dots, F_m]$, initial assignment $\mathcal{A}$, $k_{\max}$, tolerance ε

Repeat until convergence or $k > k_{\max}$:

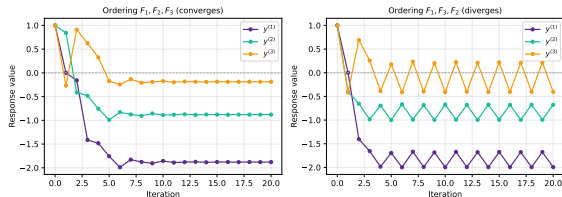
- ① $k \leftarrow k + 1$
- ② $\mathcal{A}_{\text{old}} \leftarrow \text{copy}(\mathcal{A})$
- ③ For each F_i : run $F_i(\mathcal{A})$ in order
- ④ Check: $\|v(\mathcal{A}) - v(\mathcal{A}_{\text{old}})\|_{\infty} \leq \varepsilon$ for all v

Return ($\mathcal{A}$, converged)

Ordering Matters!

A topological sort of the dependency graph (if acyclic) gives an optimal ordering. For cyclic graphs, ordering affects convergence.

Gauss-Seidel MDA — Effect of Disciplinary Ordering



Ordering F_1, F_2, F_3 converges; F_1, F_3, F_2 oscillates
(Example 24.2, $x = 1$)

Python: Gauss-Seidel MDA

```
import copy, numpy as np

def gauss_seidel_mda(Fs, A, k_max=100, eps=1e-4):
    """
    Fs      : list of disciplinary analysis functions (each mutates A in-place)
    A       : assignment dict {'x': ..., 'y1': ..., ...}
    k_max   : maximum iterations
    eps     : relative tolerance
    Returns : (A, converged)
    """
    k, converged = 0, False
    while not converged and k <= k_max:
        k += 1
        A_old = copy.deepcopy(A)
        for F in Fs:
            F(A)                                # update assignment in place
        # Check convergence on all numeric values
        converged = all(
            abs(A[v] - A_old[v]) / (abs(A_old[v]) + 1e-12) <= eps
            for v in A if isinstance(A[v], (int, float, np.floating))
        )
    return A, converged

# Run Example 24.2: x=1, ordering F1->F2->F3
A = {'x': 1.0, 'y1': 1.0, 'y2': 1.0, 'y3': 1.0}
A_sol, ok = gauss_seidel_mda([F1, F2, F3], A)
print(f"Converged: {ok}, y1={A_sol['y1']:.4f}, y2={A_sol['y2']:.4f}, y3={A_sol['y3']:.4f}")
```

MDO Architectures Overview

The general MDO problem can be written as:

$$\min_{\mathbf{x}} f(\mathcal{A}) \quad \text{s.t.} \quad \mathcal{F}_i(\mathcal{A}) = \mathbf{0}, \quad i = 1, \dots, m, \quad \mathcal{A} \in \mathcal{X}$$

Different **architectures** differ in *how* they handle the coupling:

MDO Architecture Comparison

Architecture	Optimizer Level	MDA Required	Parallel Disciplines	Variable Count	Notes
MDF	Single	Yes (inner loop)	No	Low (n)	Simple; MDA cost dominates
Sequential	Hierarchical	Per discipline	No	Low-Med	Good for loosely coupled
IDF	Single	No	Yes	High ($n + \sum n_i$)	Coupling as constraint
CO	Bi-level	No	Yes	High	Each discipline independent
SAND	Single (large)	No (residuals)	Yes	Very high	Needs analytic gradients

Single-level architectures:

- MDE, IDF, SAND — one optimizer

Hierarchical / distributed:

- Sequential, CO — multiple levels

Running Example: Ride-Sharing Company (Example 24.3)

A ride-sharing company simultaneously designs:

- **v**: vehicle (geometry, drivetrain, battery)
- **s**: sensor package (safety requirements)
- **r**: routing strategy (algorithm)
- **p**: pricing scheme

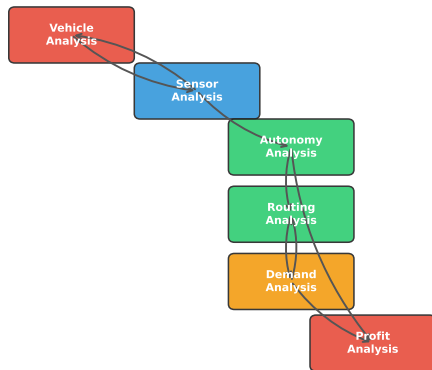
Objective: maximize profit ($= -f$)

Analyses: Vehicle, Sensor, Autonomy, Routing, Demand, Profit

Cyclic Couplings

Vehicle $\leftrightarrow$ Sensor (tight), Routing $\leftrightarrow$ Demand (tight).
Used throughout the chapter to demonstrate all architectures.

Ride-Sharing Problem: Dependency Graph



Dependency diagram for the ride-sharing problem

Multidisciplinary Design Feasible (MDF)

MDF Formulation

$$\min_{\mathbf{x}} f(\text{MDA}(\mathbf{x})) \quad \text{s.t.} \quad \mathbf{x} \in \mathcal{X}$$

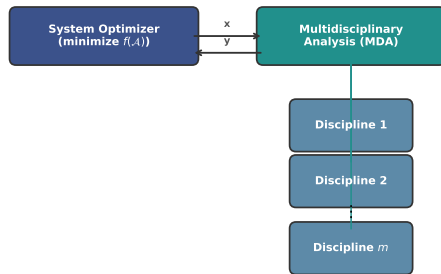
Interdisciplinary compatibility is satisfied *automatically* by running a full MDA at every optimizer call.

Procedure:

- 1 Optimizer proposes design $\mathbf{x}$
- 2 Inner MDA loop (Gauss-Seidel) solves for compatible $\mathbf{y}$
- 3 Evaluate objective f on converged $\mathcal{A}$
- 4 Optimizer updates $\mathbf{x}$; repeat

Properties

Multidisciplinary Design Feasible (MDF) Architecture



MDF architecture: single optimizer calls MDA as a black box

Python: MDF Implementation

```
import copy, numpy as np
from scipy.optimize import minimize

def mda(x_val, Fs, y_init=None, k_max=200, eps=1e-6):
    """Run a full MDA for design variable x and return converged assignment."""
    A = {'x': x_val}
    A.update(y_init or {'y1': 0.0, 'y2': 0.0, 'y3': 0.0})
    A, converged = gauss_seidel_mda(Fs, A, k_max=k_max, eps=eps)
    if not converged:
        raise RuntimeError('MDA did not converge')
    return A

def objective_mdf(x_val):
    """Objective function under MDF: run MDA, then evaluate f."""
    A = mda(x_val[0], [F1, F2, F3])
    return A['y1']**2 + A['y2']**2 + A['y3']**2    # example objective

# MDF optimization (design variable x in [-5, 5])
result = minimize(objective_mdf, x0=[0.0],
                  bounds=[(-5.0, 5.0)],
                  method='L-BFGS-B',
                  options={'ftol': 1e-9})
print(f"x* = {result.x[0]:.4f},  f* = {result.fun:.4f}")
```

MDF Applied to Ride-Sharing (Example 24.4)

For the ride-sharing problem, the MDF formulation is:

$$\begin{aligned} \min_{\mathbf{v}, \mathbf{s}, \mathbf{r}, \mathbf{p}} \quad & f(\text{MDA}(\mathbf{v}, \mathbf{s}, \mathbf{r}, \mathbf{p})) \\ \text{s.t.} \quad & [\mathbf{v}, \mathbf{s}, \mathbf{r}, \mathbf{p}] \in \mathcal{X} \\ & \text{MDA}(\cdot) \text{ converged} \end{aligned}$$

The optimizer *only* controls global design variables. The MDA handles all interdisciplinary coupling internally.

Advantage

Conceptually simple; reduces MDO to a conventional single-discipline optimization problem.

MDF Optimization Problem (General)

$$\begin{aligned} \min_{\mathbf{x}} \quad & f(\mathbf{x}, \mathbf{y}^*) \\ \text{s.t.} \quad & \mathbf{x} \in \mathcal{X}, \quad \mathbf{y}^* = \text{MDA}(\mathbf{x}) \end{aligned}$$

where $\mathbf{y}^* = [\mathbf{y}^{(1)*}, \dots, \mathbf{y}^{(m)*}]$ is the fully-compatible response vector.

Limitation

Weight sharing across disciplines not exploited; MDA convergence check added as hard constraint.

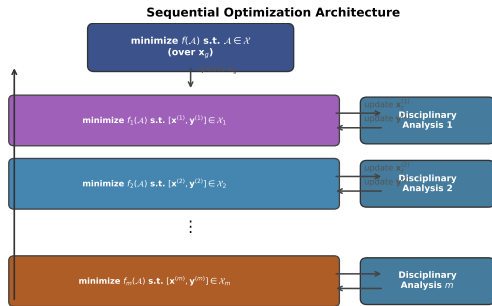
Sequential Optimization Architecture

Idea

Decompose into **discipline-level subproblems**, each optimizing a local objective f_i subject to discipline-specific constraints, one after another.

Procedure:

- 1 Top-level optimizer updates global $\mathbf{x}_g$
- 2 Subproblem 1: $\min_{\mathbf{x}_\ell^{(1)}} f_1(\mathcal{A})$ s.t. $[\mathbf{x}^{(1)}, \mathbf{y}^{(1)}] \in \mathcal{X}_1$; run Analysis 1
- 3 Subproblem 2: $\min_{\mathbf{x}_\ell^{(2)}} f_2(\mathcal{A})$ s.t. $[\mathbf{x}^{(2)}, \mathbf{y}^{(2)}] \in \mathcal{X}_2$; run Analysis 2
- 4 $\vdots$
- 5 Subproblem m : $\min_{\mathbf{x}_\ell^{(m)}} f_m(\mathcal{A}) \dots$
- 6 Return to step 1 until convergence



Sequential Optimization: Properties and Example 24.5

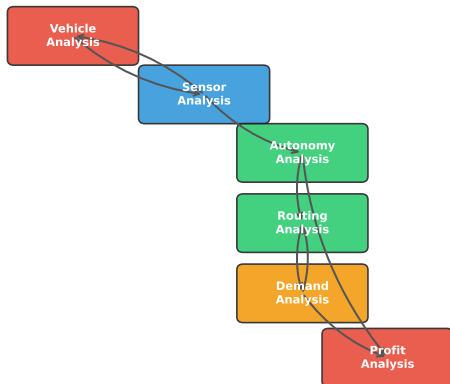
Properties

- ✓ Each subproblem uses domain-specific solvers
- ✓ Modular decomposition
- ✓ Can exploit parallel execution *within* each subproblem
- ✗ Not guaranteed to find global MDO optimum
- ✗ Poor handling of tightly-coupled disciplines
- ✗ No guarantee of interdisciplinary compatibility

Ride-Sharing (Example 24.5)

Vehicle and sensor systems are *tightly coupled*: changes by vehicle sub-problem are addressed by sensor sub-problem, but the reverse must wait for the next outer iteration. Local variables such as drive train are in $\mathbf{y}_1$; shared vehicle capacity is in $\mathbf{y}$

Ride-Sharing Problem: Dependency Graph



Sequential architecture applied to ride-sharing:
Vehicle → Sensor → Autonomy → Routing →
Demand → Profit

Individual Discipline Feasible (IDF) Architecture

Key Idea

Replace the MDA inner loop with **coupling variables** $\mathbf{c}^{(i)}$ that are *optimizer-controlled copies* of the response $\mathbf{y}^{(i)}$.

IDF Formulation

$$\min_{\mathbf{x}, \mathbf{c}} f(\mathbf{v}, \mathbf{s}, \mathbf{r}, \mathbf{p}, \mathbf{c})$$

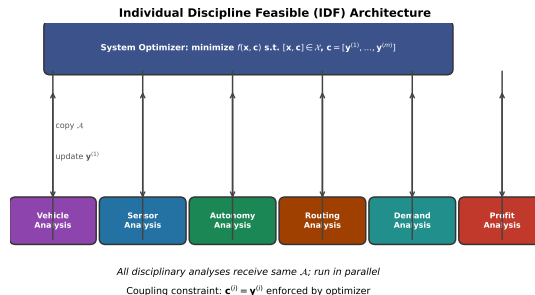
$$\text{s.t. } [\mathbf{x}, \mathbf{c}] \in \mathcal{X}$$

$$\mathbf{c}^{(i)} = \mathbf{y}^{(i)}(\mathbf{x}, \mathbf{c}) \quad \forall i$$

Coupling equality constraints enforce compatibility.

Properties

- ✓ Disciplines run *in parallel* (no MDA)



IDF applied to ride-sharing: all disciplines receive same $\mathcal{A}$; run in parallel; coupling enforced by optimizer

Python: IDF Formulation

```
import numpy as np
from scipy.optimize import minimize

# IDF: coupling variables  $c = [y1\_c, y2\_c, y3\_c]$  are decision variables
#  $x\_idf = [x, y1\_c, y2\_c, y3\_c]$ 

def idf_objective(x_idf):
    x, y1c, y2c, y3c = x_idf
    # Objective depends on coupling copies
    return y1c**2 + y2c**2 + y3c**2

def idf_constraints(x_idf):
    x, y1c, y2c, y3c = x_idf
    # Disciplinary residuals:  $y^{(i)} - c^{(i)} = 0$ 
    # Each discipline uses current assignment with coupling copies
    A = {'x': x, 'y1': y1c, 'y2': y2c, 'y3': y3c}
    r1 = (A['y2'] - A['x']) - y1c # F1 residual
    r2 = np.sin(A['y1'] + A['y3']) - y2c # F2 residual
    r3 = np.cos(A['x'] + A['y2'] + A['y1']) - y3c # F3 residual
    return [r1, r2, r3]

constraints = [{'type': 'eq', 'fun': idf_constraints}]
x0 = [1.0, 1.0, 1.0, 1.0]
result = minimize(idf_objective, x0,
                  constraints=constraints,
                  method='SLSQP',
                  options={'ftol': 1e-9, 'maxiter': 1000})
```

Collaborative Optimization (CO)

Idea

A **bi-level** architecture:

- *System-level* optimizer controls global targets $\mathcal{A}_g$
- Each discipline has its own *sub-optimizer* that minimizes a coupling objective $J^{(i)}$

System-Level Problem

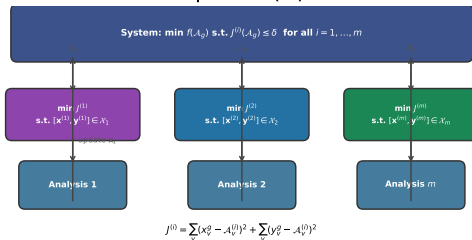
$$\min_{\mathcal{A}_g} f(\mathcal{A}_g) \quad \text{s.t.} \quad J^{(i)}(\mathcal{A}_g) \leq \delta, \forall i$$

Disciplinary Sub-Problem i

$$\min_{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}} J^{(i)}(\mathcal{A}_g) \quad \text{s.t.} \quad [\mathbf{x}^{(i)}, \mathbf{y}^{(i)}] \in \mathcal{X}_i$$

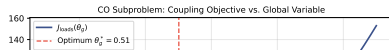
$$J^{(i)} = \sum_{\mathbf{y}} (\mathbf{y}^g - \mathcal{A}_g^{(i)})^2 + \sum_{\mathbf{x}} (\mathbf{x}^g - \mathcal{A}_g^{(i)})^2$$

Collaborative Optimization (CO) Architecture



Coupling: each subproblem drives local values to match system-level targets

CO: system-level optimizer specifies shared targets $\mathcal{A}_g$; sub-optimizers minimize coupling mismatch



Python: Collaborative Optimization Sub-Problem

```
import numpy as np
from scipy.optimize import minimize

def make_co_subproblem(i, A_g, discipline_fn, local_init):
    """
    Build and solve the CO sub-problem for discipline i.
    A_g      : system-level target assignment dict
    discipline_fn : function A -> A (updates response y^(i))
    local_init   : initial local variables [x_l, y_c]
    """
    x_g = A_g['x'] # shared global variable target

    def J_sub(local_vars):
        A = dict(A_g) # start from global targets
        A.update({'y%d' % (i+1): local_vars[0]}) # local response
        discipline_fn(A)
        y_local = A['y%d' % (i+1)]
        # Coupling objective: drive local to match global targets
        return (local_vars[0] - A_g.get('y%d_g' % (i+1), 0.0))**2

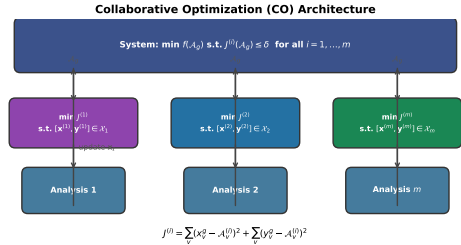
    result = minimize(J_sub, local_init, method='L-BFGS-B')
    return result.x, result.fun

# System-level: vary A_g to drive J^(i) -> 0
def system_objective(x_g_val, threshold=1e-4):
    A_g = {'x': x_g_val[0], 'y1_g': 0.0, 'y2_g': 0.0, 'y3_g': 0.0}
    # Each discipline sub-optimizes in parallel
```

CO Applied to Ride-Sharing (Example 24.7)

The CO architecture applied to the ride-sharing problem:

- System targets: $\mathcal{A}_g = \{\mathbf{v}_g, \mathbf{s}_g, \mathbf{r}_g, \mathbf{p}_g, \theta_g, M_g, \dots\}$
- Vehicle sub-optimizer minimizes J_v , sensor sub-optimizer minimizes J_s , etc.
- System optimizer minimizes $-\text{profit}$ subject to $J^{(i)} \leq \delta$



Coupling: each subproblem drives local values to match system-level targets

Spring-Pendulum CO (Exercise 24.6)

Two sub-problems:

$$\min_{k, M} J_{\text{disp}} = (k_g - k)^2 + (M_g - M)^2 + (\theta_{\text{disp}}(k_g, M_g) - \theta)^2$$

$$\min_{\theta_g} J_{\text{loads}} = (\theta_g - \theta)^2 + (F_{\text{loads}}(\theta_g) - M)^2$$

$$\text{System: } \min_{k_g, \theta_g, M_g} k_g \quad \text{s.t. } J_{\text{disp}} = 0, J_{\text{loads}} = 0$$

Primary Advantage

Discipline teams can independently apply domain-specific methods (e.g. topology

Simultaneous Analysis and Design (SAND)

Idea

Treat *both* design variables $\mathbf{x}$ and response variables $\mathbf{y}$ as **free optimization variables**. Disciplinary equations are enforced as equality constraints (residuals).

SAND Formulation

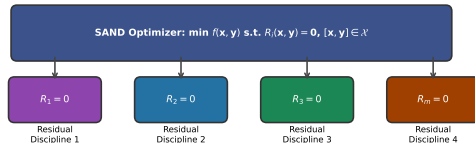
$$\begin{aligned} \min_{\mathbf{x}, \mathbf{y}} \quad & f(\mathbf{x}, \mathbf{y}) \\ \text{s.t.} \quad & R_i(\mathbf{x}, \mathbf{y}) = \mathbf{0}, \quad i = 1, \dots, m \\ & [\mathbf{x}, \mathbf{y}] \in \mathcal{X} \end{aligned}$$

where $R_i(\mathbf{x}, \mathbf{y}) \equiv \mathcal{F}_i(\mathbf{x}, \mathbf{y}) - \mathbf{y}^{(i)}$ is the disciplinary residual.

Properties

- ✓ Single monolithic optimizer — no inner MDA loop

Simultaneous Analysis and Design (SAND) Architecture



All design $\mathbf{x}$ and response $\mathbf{y}$ variables passed simultaneously to optimizer

Residuals $R_i(\mathbf{x}, \mathbf{y}) = \mathbf{0}$ enforced as equality constraints

Enables gradient-based methods; disciplinary analyses run once per iteration

Key Distinction from MDF

MDF satisfies residuals at *every function evaluation*. SAND satisfies residuals only at *convergence*. Intermediate iterates are infeasible.

Python: SAND Formulation

```
import numpy as np
from scipy.optimize import minimize

# SAND: x_sand = [x, y1, y2, y3]
# Residuals R_i must be zero at solution

def sand_objective(z):
    x, y1, y2, y3 = z
    return y1**2 + y2**2 + y3**2    # minimize sum of squares

def sand_residuals(z):
    x, y1, y2, y3 = z
    # R_i = F_i(A) - y_i (disciplinary residuals)
    R1 = (y2 - x) - y1                # F1 residual
    R2 = np.sin(y1 + y3) - y2         # F2 residual
    R3 = np.cos(x + y2 + y1) - y3     # F3 residual
    return [R1, R2, R3]

def sand_jacobian_residuals(z):
    """Analytic Jacobian of residuals wrt z = [x, y1, y2, y3]."""
    x, y1, y2, y3 = z
    # dR1/dz = [-1, -1, 1, 0]
    # dR2/dz = [0, cos(y1+y3)-1, -1, cos(y1+y3)]
    # dR3/dz = [-sin(x+y2+y1), -sin(x+y2+y1), -sin(x+y2+y1)-1, -1]
    s2 = np.cos(y1 + y3)
    s3 = -np.sin(x + y2 + y1)
    return np.array([[-1, -1, 1, 0],
```

SAND Applied to Ride-Sharing

The SAND formulation for the ride-sharing example:

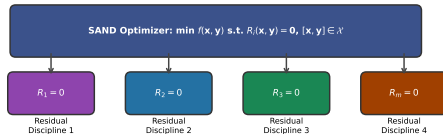
$$\begin{aligned} \min_{\mathbf{v}, \mathbf{s}, \mathbf{r}, \mathbf{p}, \mathbf{y}^{(v)}, \dots, \mathbf{y}^{(p)}} \quad & f(\mathbf{v}, \mathbf{s}, \mathbf{r}, \mathbf{p}, \mathbf{y}) \\ \text{s.t.} \quad & R_{\text{vehicle}}(\mathbf{v}, \mathbf{y}) = 0 \\ & R_{\text{sensor}}(\mathbf{s}, \mathbf{y}) = 0 \\ & \vdots \\ & R_{\text{profit}}(\mathbf{p}, \mathbf{y}) = 0 \end{aligned}$$

All design and response variables optimized *simultaneously*.

Practical Use

SAND is most effective when used with gradient-based solvers (e.g., interior-point, SQP) and adjoint-based sensitivities, e.g., in aerodynamic shape optimization.

Simultaneous Analysis and Design (SAND) Architecture



All design $\mathbf{x}$ and response $\mathbf{y}$ variables passed simultaneously to optimizer

Residuals $R_i(\mathbf{x}, \mathbf{y}) = 0$ enforced as equality constraints

Enables gradient-based methods; disciplinary analyses run once per iteration

Residual Form

For discipline i :

$$R_i(\mathbf{x}, \mathbf{y}) \equiv \mathbf{y}^{(i)} - \mathcal{F}_i(\mathbf{x}, \mathbf{y}) = 0$$

Total constraints: $\sum_{i=1}^m n_i$ equality constraints

Spring-Pendulum Numerical Example (Exercises 24.4–24.6)

Problem Setup:

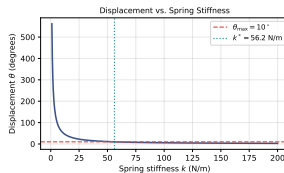
Minimize spring stiffness k such that displacement $\theta \leq \theta_{\max}$.

Two disciplinary analyses:

- **Loads:** $M = mg\ell \cos(\theta)$ (moment from gravity)
- **Displacement:** $\theta = M/k$ (spring restoring)

Parameters: $m = 1 \text{ kg}$, $\ell = 1 \text{ m}$, $g = 9.81 \text{ m/s}^2$,
 $\theta_{\max} = 10^\circ$

Spring-Pendulum MDF Example (Exercise 24.4)



MDF Solution

$$\min_{k>0} f(\text{MDA}(k)) \quad \text{s.t.} \quad \theta \leq \theta_{\max}$$

Result: $k^* \approx 55.353 \text{ N/m}$

IDF Solution (Exercise 24.5)

Python: Spring-Pendulum MDF Solution

```
import numpy as np
from scipy.optimize import minimize_scalar, minimize

m, ell, g = 1.0, 1.0, 9.81
theta_max = np.deg2rad(10.0)      # 10 degrees in radians

def mda_pendulum(k, tol=1e-10, max_iter=100):
    """Gauss-Seidel MDA for spring-pendulum system."""
    theta = theta_max / 2          # initial guess
    for _ in range(max_iter):
        M = m * g * ell * np.cos(theta) # loads analysis
        theta_new = M / k           # displacement analysis
        if abs(theta_new - theta) < tol:
            return theta_new, M, True
        theta = theta_new
    return theta, M, False

def mdf_objective(k):
    theta, M, converged = mda_pendulum(k)
    penalty = max(0, theta - theta_max)**2 * 1e6
    return k + penalty              # minimize k subject to theta <= theta_max

# Bounded scalar optimization
result = minimize_scalar(mdf_objective, bounds=(1.0, 1000.0),
                        method='bounded', options={'xatol': 1e-8})

k_star = result.x
theta_star, M_star, _ = mda_pendulum(k_star)
```

Summary I

Summary II

Chapter 24 Key Takeaways

- ① **MDO** addresses optimization problems involving multiple interacting disciplines, where solving each discipline in isolation leads to suboptimal system-level solutions.
- ② **Disciplinary analysis** $\mathcal{F}_i$ takes a shared assignment $\mathcal{A}$ and updates the response variables $\mathbf{y}^{(i)}$. Running analyses in cycles gives the **Gauss-Seidel MDA**; ordering affects convergence.
- ③ **Interdisciplinary compatibility**: assignment $\mathcal{A}$ is feasible iff all disciplinary residuals $R_i = 0$. Dependency graphs (with cycles $\Rightarrow$ MDA required).
- ④ Five main architectures:
 - **MDF**: black-box MDA at every function evaluation; simplest
 - **Sequential**: hierarchical, discipline-by-discipline; modular
 - **IDF**: coupling variables as optimizer decisions; parallel disciplines
 - **CO**: bi-level; each discipline has own sub-optimizer; maximum autonomy
 - **SAND**: all variables free; residuals as constraints; one MDA at convergence
- ⑤ Trade-offs:
 - MDF: low variable count, expensive (MDA per eval)
 - IDF/SAND: large variable count, disciplinary parallelism
 - CO: disciplinary autonomy, but complex bi-level structure

Exercises

Exercise 24.1

Provide an example of a real-world multidisciplinary design optimization problem with at least three disciplines. Draw the dependency diagram. Explain which analyses are independent and which require iteration.

Exercise 24.2

Consider the three-discipline system from Example 24.2:

$$y^{(1)} = y^{(2)} - x, \quad y^{(2)} = \sin(y^{(1)} + y^{(3)}), \quad y^{(3)} = \cos(x + y^{(1)} + y^{(2)})$$

Show that the ordering F_1, F_3, F_2 does not converge for $x = 1$. What does topological sorting suggest?

Exercise 24.4 (Spring-Pendulum — MDF)

Formulate and solve the spring-pendulum problem under MDF with $m = 1 \text{ kg}$, $\ell = 1 \text{ m}$, $g = 9.81 \text{ m/s}^2$, $\theta_{\max} = 10^\circ$. Show that $k^* \approx 55.353 \text{ N/m}$.

Exercise 24.5 (Spring-Pendulum — IDF)

Further Reading & References

-  M. J. Kochenderfer and T. A. Wheeler, *Algorithms for Optimization*, 2nd ed., MIT Press, 2026.
-  J. R. R. A. Martins and A. B. Lambe, “Multidisciplinary design optimization: A survey of architectures,” *AIAA Journal*, 51(9):2049–2075, 2013.
-  E. J. Cramer et al., “Problem formulation for multidisciplinary optimization,” *SIAM Journal on Optimization*, 4(4):754–776, 1994.
-  R. D. Braun and I. M. Kroo, “Use of the collaborative optimization architecture for launch vehicle design,” in *AIAA/NASA MDO Symposium*, 1996.